

```

private static void checkBaseClass(Class currentClass) {
    if(currentClass != null) {
        Class superClass = currentClass.getSuperClass();
        checkBaseClass(superClass);
    }
    if(superClass != null) {
        System.out.printf(" %s" -> "%s" \n ",
            superClass.getName(), currentClass.getName());
    }
    checkInterfaces(currentClass);
}

public static void main(String []args) {
    if(args.length != 1) {
        System.err.println("Enter the name of the class as parameter");
        System.exit(-1);
    }

    System.out.println("digraph LinkedListHierarchy { ");
    System.out.println(" \t node[shape = box]");
    System.out.println(" \t edge[arrowtail=empty]");

    dir="back");
    Class currentClass = null;
    try {
        currentClass = Class.forName(args[0]);
    } catch (ClassNotFoundException cnfe) {
        System.out.println("enter the fully qualified class name as
argument");
        System.exit(-1);
    }
    checkBaseClass(currentClass);
    System.out.println(")");
}

```

The `currentClass.getInterfaces()` method returns the array of interfaces that the class implements; we use it in the `checkInterfaces` method. We want to differentiate the interface inheritance from the class inheritance, so we use a dotted line for interfaces (using the `[shape = dotted]` attribute for the edge). After compiling the class, we run the following command to pipe the output of the Java program directly to the `dot` program, which will create the graph.

```
$ java FullClassHierarchy java.util.LinkedList | dot -Tjpeg -o LLHierarchy.jpeg
```

The graph that is output into `LLHierarchy.jpeg` is shown

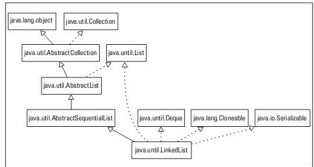


Figure 3: Inheritance hierarchy of `java.util.LinkedList`

in Figure 3.

Yes, this indeed is a nice diagram of the whole hierarchy of `java.util.LinkedList`, and it's impressive because we wrote just a few dozen lines of code to achieve this! Now you can graph the hierarchy of any of the classes in the Java core package, using this program!

Getting acquainted with Graphviz will take some time. We'll have to learn the syntax of `dot`—believe me, it is a very simple language to learn, and it takes only a few minutes to get started writing `dot` programs! Once you try creating complex diagrams using Graphviz, you'll start appreciating its capabilities. I would strongly recommend you to get started, and try exploring it yourself. **END** 

Resources

Useful Links:

- The Graphviz homepage is at <http://www.graphviz.org>.
- The Wikipedia page for Graphviz gives a nice overview of Graphviz: <http://en.wikipedia.org/wiki/Graphviz>.
- The resources section of the Graphviz website, <http://graphviz.org/Resources.php> lists GUIs for Graphviz, various language bindings (Perl, Java, Python, ...), tools that use Graphviz for generating graphs, etc.

Installing Graphviz: You can download RPMs for Red Hat, CentOS or Fedora, or debs for Ubuntu, from <http://www.graphviz.org/Download.php> or build it from source. I installed Graphviz with an RPM, and it was very smooth.

By: S G Ganesh

The author is a research engineer in Siemens (Corporate Technology), Bangalore. His latest book, 'Cracking the C, C++ and Java Interview', published by Tata McGraw-Hill, New Delhi, is a best seller. You can reach him at sgganesh@gmail.com.

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09326087210. info@technoinfotech.com